CS315 · ALGORITHMS ANALYSIS & DESIGN

Theory Review

النظري المهم للميد في ملف واحد

محارب

محتوى الملف

01 صح وخطأ واختيارات الميدات

02 L1: تحليل الخوارزميات

03 L2: Divide and Conquer

04 L3: Hash Tables

05 L4: Dynamic Programming

06 L5: Greedy

07 L6–7: B-Trees

08 اختبر نفسك

01 صح وخطأ واختيارات الميدات

الجملة	الجواب	الميد
The code length in Huffman codes depends on the frequency of occurrence of characters.	**T**	451 462 472
Greedy algorithm can solve all the dynamic programming problems.	**F**	451 462 472
$T(n) = T(\frac{n}{2}) + T(\frac{n}{2}) + T(\frac{MERGE}{2})$ for MERGESORT.	**F**	451 462 472
An algorithm which uses the past results to find new results is ...	**DP**	451 462 472
The recursive activity-selection algorithm takes O(n log n).	**قيمة ثانية: Θ(n)**	451 462
To compare two algorithms, the same HW/SW must be used without any consideration of inputs.	**F**	442 472
Theoretical analysis evaluates speed independent of the HW/SW environment.	**T**	442 472
The total of primitive operations represents the worst case.	**T**	442
DP takes far less time than methods that don't use overlapping sub-problems.	**T**	462

462	**أكبر تماماً**	f(n) is ω(g(n)) if f is asymptotically … g(n).
462	**O(1)**	Best case of linear search.
462	**binary search**	Which function is O(log n)?
462	**O(n²)**	Two nested loops of n each.
461	**F**	An algorithm is a disorganized set of steps in an unspecified time.
461	**T**	Running time typically grows with the input size.
461	**F (n²)**	Insertion sort sorts n elements in time about n.
461	**T**	Main operations of a dictionary: search, insert, delete.
461	**F**	Many optimal solutions but only one feasible solution.
461	**إدخال كثير وبحث قليل**	The log file is most effective when …
461	**characterize the structure**	First step of a DP algorithm.
461	**O(n log n)**	Running time of Huffman encoding.

سؤال 461: «Fractional knapsack في O(n)» نموذج الحل ما هو مؤكد فيه. لو العناصر مو مرتبة نحتاج ترتيب `O(n log n)`، فالجواب الأقرب F.

02 L1: تحليل الخوارزميات

المصطلح	المعنى
Algorithm	A step-by-step procedure for solving a problem in a finite amount of time.
Worst case	اللي نحلله دايماً: أسهل في التحليل، ومهم لتطبيقات مثل games و finance و robotics.
حدود التجارب	لازم نبرمج الخوارزمية، والنتيجة ممكن ما تمثل مدخلات ثانية، ولازم نفس الـ HW/SW ونفس المدخلات.
Theoretical analysis	وصف عالي المستوى، دالة في n، كل المدخلات، ومستقل عن الـ HW/SW.
RAM model	CPU واحد، وذاكرة مرقمة غير محدودة، والوصول لأي خانة بوقت ثابت.
Primitive operations	تقييم تعبير، تعيين، indexing، استدعاء دالة، return. كل وحدة وقت ثابت.
arrayMax	Best `7n - 2`، Worst `5n`، ومع سطر Temp `7n`.
تغيير الجهاز	يغير الثابت بس، مو الـ growth rate.
prefixAverages	الأولى `O(n²)`، والثانية (running sum) `O(n)`.

الرمز	المعنى
O	أقل من أو يساوي (upper bound)
Ω	أكبر من أو يساوي (lower bound)
Θ	يساوي
o	أقل تماماً
ω	أكبر تماماً

f(n) is O(g(n)) if there are positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for $n \geq n_0$.
Use the smallest class and the simplest expression: 2n is O(n), 3n + 5 is O(n).

L2: Divide and Conquer 03

النقطة	وش تحفظ
الخطوات	Divide → Conquer → Combine
Loop invariant	Initialization · Maintenance · Termination
Insertion sort	Best $O(n)$ (مرتبة)، Worst $O(n^2)$ (بالعكس). المقارنة أغلى من الـ swap.
Merge sort	$O(n \log n)$ في كل الحالات. يحتاج مصفوفة إضافية وخطوة merge.
المعادلة	$T(n) = T(\frac{n}{2}) + T(\frac{n}{2}) + T(MERGE)$
شجرة الاستدعاء	$\log(n) + 1$ مستوى، والمستوى t فيه 2^t مسألة حجمها $\frac{n}{2^t}$، وشغل كل مستوى خطي.
Quick sort	يقسم حول pivot (ممكن غير متساوي)، ما يحتاج مصفوفة إضافية ولا merge. Worst $O(n^2)$.

L3: Hash Tables 04

النقطة	وش تحفظ
Log file	find ، `O(1)` insert و remove `O(n)` . ينفع لما الإدخال كثير والبحث قليل.
Hash function	`h(x) = h₂(h₁(x))` : hash code map ثم compression map.
متى تكون "good"؟	توزع المفاتيح بشكل عشوائي ومتساوي فالتصادم قليل، وسريعة.
Hash code maps	Memory address، Integer cast، Component sum، Polynomial accumulation (للـ strings).
Compression	Division `y mod N` (N أولي)، و MAD بشرط `(ay + b) mod N` . `a mod N ≠ 0`
Chaining	كل خانة فيها linked list. بسيطة بس تحتاج ذاكرة برا الجدول.
Linear probing	open addressing، الخانة اللي بعدها. عيبها تتكتل العناصر.
Double hashing	`(i + j·d(k)) mod N` ، و `d(k) = q − (k mod q)` ، و d ما تكون صفر، و N أولي.
AVAILABLE	علامة مكان العنصر المحذوف عشان البحث ما يوقف غلط.
الأداء	Worst `O(1)` ، Expected `O(n)` (كل المفاتيح تتصادم). `α = N/M` ، والـ probes المتوقعة `1/(1 − α)` . Rehashing يضاعف حجم الجدول.

L4: Dynamic Programming 05

النقطة	وش تحفظ
الفكرة	نحل المسائل الصغيرة ونخزن نتايجها ولا نعيد حسابها. Bottom-up. مبدأ Bellman: كل optimal policy من optimal sub-policies.
الخاصيتين	Optimal sub-structure · Overlapping sub-problems
الفرق عن D&C	في D&C المسائل مستقلة وما تتكرر.
الخطوات الأربع	1. Characterize 2. Recursively define 3. Compute (bottom-up) 4. Construct
التطبيقات	Matrix chain · LCS · Optimal BST
Fibonacci	DP، $O(2^n)$ Recursive $O(n)$.
Matrix chain	تكلفة $p \cdot q \cdot r$. الهدف أقل عدد ضرب. $O(n^3)$.
LCS	تطابق: القطر + 1. اختلاف: max(فوق، يسار). الطول في آخر خانة. $O(mn)$.

L5: Greedy 06

النقطة	وش تحفظ
الفكرة	ياخذ أفضل خيار الحين وما يرجع عنه. Top-down. ما يضمن الـ optimum دايماً.
الخاصيتين	Greedy-choice property · Optimal substructure
Activity selection	نرتب حسب وقت النهاية وناخذ اللي يخلص أول. `θ(n)` لو مرتبة.
Fractional knapsack	Greedy يعطي الأمثل: نرتب حسب $\frac{v}{w}$. `O(n log n)` بسبب الترتيب.
0-1 knapsack	Greedy ممكن يفشل (يبقى مكان فاضي)، والحل DP.
Huffman	نجمع أصغر تكرارين كل مرة. يسار 0 ويمين 1. الأكثر تكرار كوده أقصر. Prefix code. `O(n log n)`.

07 L6–7: B-Trees

النقطة	وش تحفظ
التعريف	شجرة متوازنة للأقراص، كل الأوراق بنفس العمق، والعقدة اللي فيها n مفتاح لها n + 1 ابن.
الحدود (t)	المفاتيح من `t − 1` لين `2t − 1`، والجذر أقل شي مفتاح واحد. t = 2 هي 2-3-4 tree.
تسمية الميد	order 5 = أكثر شي 4 مفاتيح، degree 3 = أكثر شي مفتاحين.
الإضافة	دايماً في ورقة. العقدة الممتلية تنقسم والوسط يطلع للأب. انقسام الجذر يزيد الارتفاع.
الحذف	1: ورقة فيها t ≤ ← نحذف. 2a/2b: predecessor أو 3a. 2c: merge. نستلف successor: من الأخ. 3b: merge مع الأخ.
التكلفة	Search `O(t log$_t$ n)` · Insert `O(t log$_t$ n)` · Delete `O(t log$_t$ n)`

08 اختبر نفسك

1. Merge sort runs in O(n²) in the worst case.
2. In double hashing, N should be a prime.
3. Chaining stores colliding items in other cells of the table.
4. Greedy always gives the optimal solution for the 0-1 knapsack.
5. In DP, sub-problems are independent and never repeat.
6. A B-tree with t = 3 can have at most 5 keys in a node.
7. Splitting a leaf increases the height of a B-tree.
8. Big-Omega gives a lower bound.
9. Insertion sort on a sorted array runs in O(n).
10. In Huffman coding, a rare character gets a shorter code.

#	الجواب	السبب
1	**F**	`O(n log n)` في كل الحالات.
2	**T**	
3	**F**	هذا open addressing. الـ chaining يستخدم linked list.
4	**F**	0-1 يحتاج DP.
5	**F**	هذا D&C. في DP المسائل تتكرر.
6	**T**	`2t - 1 = 5`
7	**F**	بس انقسام الجذر.
8	**T**	
9	**T**	
10	**F**	الأكثر تكرار كوده أقصر.